Whack-A-Mole Light-Up Button Game
Final Project Report
ENG SCI-50
Lyat Melese, Nimet Yesil, Victoria Zambrano, Anya Zhang

Abstract

- Taking inspiration from Whack a Mole – made portable, button version for players to enjoy
- Integration of different game components with internal circuits + game processes
- Game features set of buttons with corresponding LEDs – the player has to press the button before the allotted time runs out, four levels of difficulty – have to get a certain number of correct buttons pressed to move on to the next level
- To tackle making the game – split project into different sub-systems + dependencies, worked on integrating smallest sub-systems + worked way up
- Ensuring user-friendly + responsive interface, requires constant information exchange between game interface + internal microcontroller to ensure that game behaves correctly to different player behaviors – ensuring enjoyable + fun gameplay

Project Background

Overall Approach

- The main components of whack-a-mole game can be separated into three main categories: physical game interface – including buttons/LEDs/buzzer; the game logic – where advancing the levels + processing what inputs + outputs; the score + game indicator – using the screen display – can figure out the score + high score
- Started with most basic component, ensured compatibility + correct functionality with the Arduino, then kept integrating different components, decision making was to pick most robust option + minimize the number of pins available
- test + using available resources
- then refine if worked + alter decision if the design didn't work
- Began with ensuring compatibility between buttons + Arduino – could correctly read which button was pressed, then integrated lights with buttons
- Used light turn on + button press as a building block – create game logic + score calculation – 4 levels to calculate how a player progresses through the game
- Finally – implemented outputs of the game – including buzzer, microSD to store the scores for each round played, LCD display to guide the player through different levels + display the scores
- When have all electronics + code working – implemented into physical interface, moved buttons + LEDs off non-solderable breadboard into protoboard

- Process required back + forth between the circuit + the Arduino interface to see what the Arduino registered, had lots of testing built into process + after the interface had been fully assembled to ensure that code was robust + nothing unexpected would happen

Circuit Components and Design

I. LEDs and Buttons
II. Buzzer
III. Screen Display

IV. MicroSD
- MicroSD – used to store the high scores, it is 32 GB because other size of microSD – often not compatible with Arduino – SD Library supports FAT32 + FAT16 file systems
- In order to connect microSD to the Arduino, has to be connected to a SD module – allows communication between the two, microSD also operates at 3.3 V while Arduino supplies 5 V, so ensures voltage compatibility
- Module has 6 pins – GND, VCC, MISO, MOSI, SCK, CS
- GND – connected to circuit negative rail, VCC provides power to the module – connected to Arduino's 5 V pin, MISO, SCK, MOSI – connected to respective Arduino pins, CS – connected to a digital pin

Project Code and Implementation

I. Physical Game Shell Design
II. Game Logic
III. LED and Button Triggering
IV. Buzzer
V. LCD Screen Display

VI. Score Memory
- Before writing to the microSD – have to first open + set up the microSD in the Arduino interface
- MicroSD stores total score from each play to a CSV file, Arduino – after has a game over indication + exits the loop that runs individual levels, opens the csv file + adds the player's total score to the CSV file
- Then, to pull the high score – does a for loop through each entry of the spreadsheet to find the highest entry – which is the high score for the game
- Arduino can then use that value + send it to the LCD display to show the player how their score compared to the highest score
- Opening + writing happens for each time the game reaches game over – so for each full round that is played, saves the score

Alternatives

- Buttons integration with the circuit – realized quickly that couldn't have each button + each LED connected to a pin – that would require 18 pins
- So decided to minimize number of pins for the buttons, deciding between using a resistor ladder and a shift register
- Could have used a shift register – but didn't have one on hand at that moment, did have resistors handy – decided to try to see if it would work + it did – relatively easy to calibrate by switching out combinations of resistors that were associated with each button
- LEDs – went with individual LEDs – does take up 9 pins, made that decision originally because wanted to save the addressable LEDs for creating a decorative frame for the game – but decided to put time elsewhere
- Also with addressable LEDs – bought them on a strip, would have had to cut them + solder wires between them, or mount them + turn on specific pixels
- Another critical design decision, not necessarily a full alternative, but rather than keep everything on the breadboard, decided to solder buttons + LEDs onto a protoboard
- that way – main parts of the game interface that the user is interacting with, soldering made connections more robust, less likely to fall out from breadboard

Limitations and Future Directions

Team Member Contributions

Bill of Materials

Arduino Code

References